

Deep Learning 1 - Backpropagation and MLP

Matúš Halák - 13626574 (UvA Student ID)

November 2025

1 Batched backpropagation

1.1 Linear Module

We analyze the gradients of the following linear module where S represents the number of batches, M represents the number of input features from the previous layer, N represents the number of features in the subsequent layer:

$$\mathbf{Y} = \mathbf{X}\mathbf{W}^T + \mathbf{B} \in \mathbb{R}^{S \times N}$$

where $\mathbf{B}$ is the bias $\mathbf{b} \in \mathbb{R}^N$ broadcast to the batch dimension ($\mathbf{B} = \mathbf{1}_S \mathbf{b}^T$), and $\mathbf{W}$ is the $N \times M$ matrix of layer weights.

Question 1a

$$\begin{aligned} \frac{\partial L}{\partial W_{nm}} &= \frac{\partial L}{\partial Y_n} \frac{\partial Y_n}{\partial W_{nm}} = \left[\frac{\partial L}{\partial Y_n} \right]^T X_{:,m} \in \mathbb{R} \\ \frac{\partial L}{\partial \mathbf{W}} &= \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{W}} = \left[\frac{\partial L}{\partial \mathbf{Y}} \right]^T \mathbf{X} \in \mathbb{R}^{N \times M} \end{aligned}$$

Question 1b

$$\frac{\partial L}{\partial \mathbf{b}} = \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{b}} = \left[\frac{\partial L}{\partial \mathbf{Y}} \right]^T \mathbf{1}_S = \sum_s \left[\frac{\partial L}{\partial \mathbf{Y}} \right]_{s,:}^T \in \mathbb{R}^N$$

Question 1c

$$\frac{\partial L}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \frac{\partial \mathbf{Y}}{\partial \mathbf{X}} = \frac{\partial L}{\partial \mathbf{Y}} \mathbf{W} \in \mathbb{R}^{S \times M}$$

1.2 Activation Module

Here we consider an element-wise activation function h :

$$Y_{ij} = h(X_{ij}) \quad \mathbf{Y}, \mathbf{X} \in \mathbb{R}^{S \times N}$$

Question 1d

$$\begin{aligned} \frac{\partial \mathbf{Y}}{\partial \mathbf{X}} &= \frac{\partial \mathbf{Y}}{\partial \mathbf{U}} \frac{\partial \mathbf{U}}{\partial \mathbf{X}} & \frac{\partial \mathbf{Y}}{\partial \mathbf{U}} &= h'(\mathbf{U}) & \frac{\partial \mathbf{U}}{\partial \mathbf{X}} &= \mathbf{I} & \frac{\partial \mathbf{Y}}{\partial \mathbf{X}} &= h'(\mathbf{X}) \quad \text{since } \mathbf{U} = \mathbf{X} \\ & & \frac{\partial L}{\partial \mathbf{X}} &= \frac{\partial L}{\partial \mathbf{Y}} \odot h'(\mathbf{X}) & & \in \mathbb{R}^{S \times N} \end{aligned}$$

1.3 Softmax and Loss Modules

Here we consider the output layer of a deep neural network with a matrix of S batches of logits for C classes $\mathbf{Z} \in \mathbb{R}^{S \times C}$ that passes through a softmax layer, followed by categorical cross-entropy loss. The final loss is the mean loss over the losses for each sample in the batch. One-hot encoded target vectors for each batch are encoded in a target matrix $\mathbf{T} \in \mathbb{R}^{S \times C}$.

$$\mathbf{Y} = \text{softmax}(\mathbf{Z}) \quad Y_{sc} = \frac{\exp(Z_{sc})}{\sum_k^C \exp(Z_{sk})} \quad L = \frac{1}{S} \sum_s^S \left(- \sum_c^C T_{sc} \log Y_{sc} \right)$$

We also make use of the closed-form solutions for the gradients of the softmax and cross-entropy modules:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{Z}} &= \mathbf{Y} \odot \left(\frac{\partial L}{\partial \mathbf{Y}} - \left(\frac{\partial L}{\partial \mathbf{Y}} \odot \mathbf{Y} \right) \mathbf{1}_{C \times C} \right) \\ \frac{\partial L}{\partial \mathbf{Y}} &= -\frac{1}{S} \frac{\mathbf{T}}{\mathbf{Y}} \end{aligned}$$

Question 1e We look for the expression $\frac{\partial L}{\partial \mathbf{Z}} = \alpha \mathbf{M}$ for $\alpha \in \mathbb{R}^+$, $\mathbf{M} \in \mathbb{R}^{S \times C}$.

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{Z}} &= \mathbf{Y} \odot \left(-\frac{1}{S} \frac{\mathbf{T}}{\mathbf{Y}} - \left(-\frac{1}{S} \frac{\mathbf{T}}{\mathbf{Y}} \odot \mathbf{Y} \right) \mathbf{1}_{C \times C} \right) \\ &= -\frac{1}{S} \mathbf{T} + \frac{1}{S} \mathbf{Y} \odot (\mathbf{T} \mathbf{1}_{C \times C}) \\ &= -\frac{1}{S} \mathbf{T} + \frac{1}{S} \mathbf{Y} \odot (\mathbf{1}_{S \times C}) \end{aligned}$$

Thus we find an elegant closed-form solution for the derivative of softmax followed by cross-entropy loss with respect to the logits:

$$\frac{\partial L}{\partial \mathbf{Z}} = \frac{1}{S} (\mathbf{Y} - \mathbf{T}) \quad \alpha = \frac{1}{S} \in \mathbb{R}^+ \quad \mathbf{M} = \mathbf{Y} - \mathbf{T} \in \mathbb{R}^{S \times C}$$

2 NumPy MLP implementation

We implement a general MLP with user-specified number and width of hidden layers in NumPy. We do this through implementation of the modules calculated above, each containing its own forward and backward pass, thus allowing us to chain the modules to arbitrary depth. For the element-wise activation function, we use the Exponential Rectified Linear Unit (ELU) function. Weights in linear layers are initialized using Kaiming initialization, and biases are initialized to 0.

We train the model using mini-batch stochastic gradient descent on the CIFAR10 classification task. Figure 1 shows the training loss curve for a 2-layer NumPy MLP with 128 hidden units, over 10 epochs with learning rate $\eta = 0.1$ and batch size 128. After 10 epochs, **the model achieved 49.4 % accuracy on the test set.**

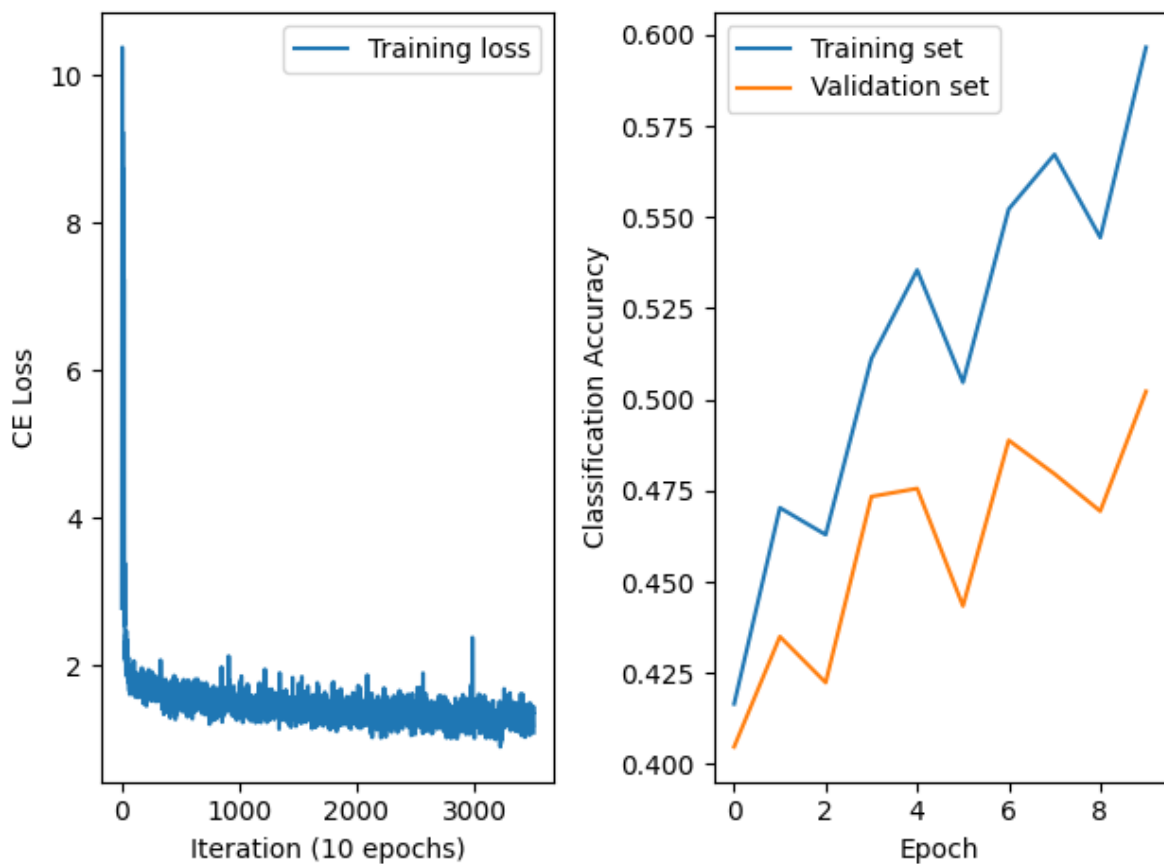


Figure 1: Learning CIFAR-10 image classification with a NumPy MLP.

3 PyTorch MLP implementation

We implement an analogue to our NumPy MLP in PyTorch, also including the option for batch normalization of linear activations in hidden layers. We train the model using with the same parameters as we used for our NumPy implementation. Figure 2 shows the training loss curve for a 2-layer PyTorch MLP with 128 hidden units, over 10 epochs with learning rate $\eta = 0.1$ and batch size 128. After 10 epochs, **the model achieved 50.0 % accuracy on the test set** (qualitatively same performance as the NumPy implementation).

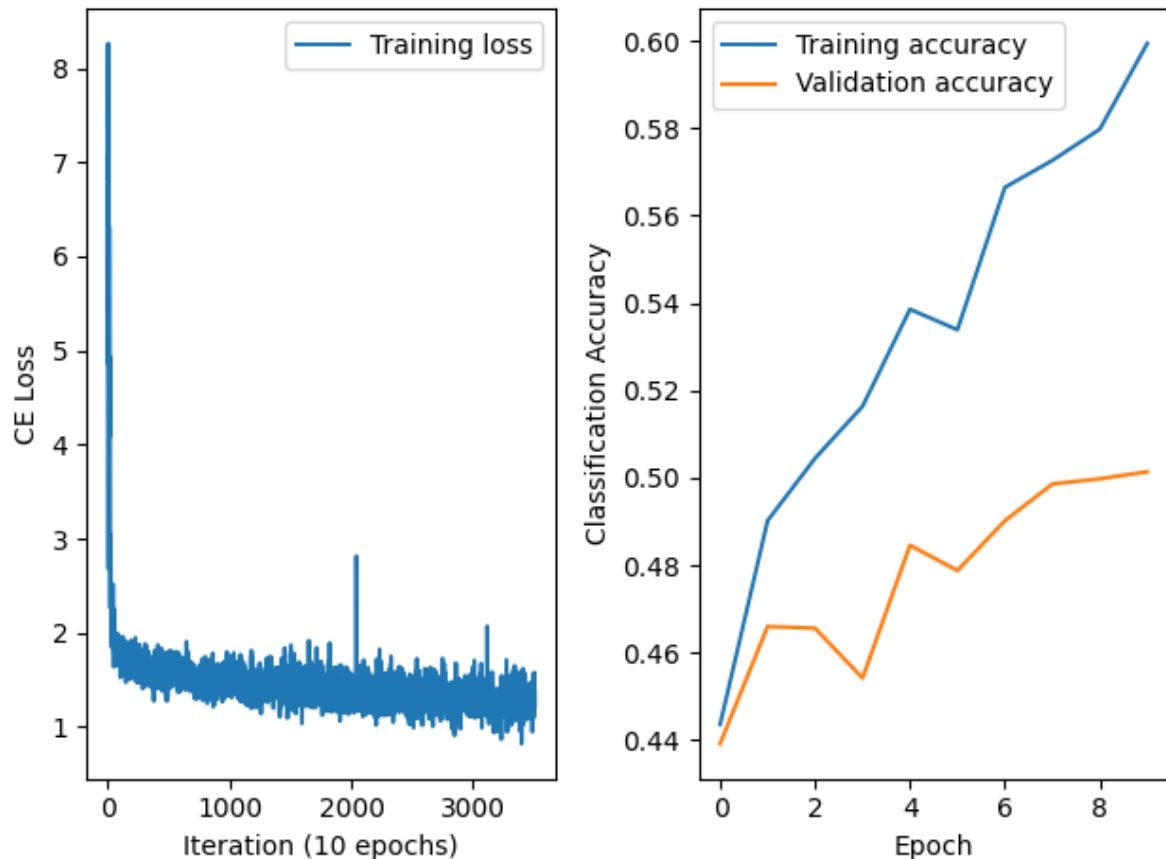


Figure 2: Learning CIFAR-10 image classification with a PyTorch MLP.

4 Optimization

Question 4a - Show that the Hessian of $f : \mathbb{R}^n \rightarrow \mathbb{R}$ in a strict local minimum $\mathbf{x}^*$ has only non-negative eigenvalues. If the Hessian $\mathbf{H} \in \mathbb{R}^{n \times n}$ of $f(\mathbf{x}^*)$ has only non-negative eigenvalues, $\mathbf{H}$ is a positive semi-definite (PSD) matrix. Thus, this question asks us to prove that $\mathbf{H}$ in a strict local minimum is PSD. For a PSD matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, it holds that $\mathbf{v}^T \mathbf{A} \mathbf{v} \geq 0 \quad \forall \mathbf{v} \in \mathbb{R}^n$. With that in mind, for some perturbation vector $\Delta \in \mathbb{R}^n$, we form the second-order Taylor expansion around $f(\mathbf{x}^*)$:

$$f(\mathbf{x}^* + \Delta) \approx f(\mathbf{x}^*) + \Delta^T \nabla f(\mathbf{x}^*) + \frac{1}{2} \Delta^T \mathbf{H} \Delta$$

Since at a local minimum $\nabla f(\mathbf{x}^*) = 0$, this reduces to

$$f(\mathbf{x}^* + \Delta) \approx f(\mathbf{x}^*) + \frac{1}{2} \Delta^T \mathbf{H} \Delta$$

From here, we notice that because $\mathbf{x}^*$ is a local minimum, any small perturbation from it increases the function value $f(\mathbf{x}^* + \Delta) - f(\mathbf{x}^*) \geq 0$. Because $f(\mathbf{x}^* + \Delta) - f(\mathbf{x}^*) \approx \frac{1}{2} \Delta^T \mathbf{H} \Delta$, we can conclude that

$$\frac{1}{2} \Delta^T \mathbf{H} \Delta \geq 0 \quad \forall \Delta \in \mathbb{R}^n$$

which proves that $\mathbf{H}$ is PSD and has all non-negative eigenvalues. In fact because $\mathbf{x}^*$ is a **strict** local minimum the Hessian is positive definite and eigenvalues are all positive if we exclude the zero vector ($\Delta^T \mathbf{H} \Delta > 0$ if $\Delta \neq \mathbf{0}_n$).

Question 4b - Why is the number of saddle points exponentially larger than the number of local minima in high-dimensional problems? For a local minimum to exist, all the eigenvalues of the Hessian must be non-negative (i.e. all the eigenvalues must have positive sign). Because the eigenvalues can have two signs (positive or negative), for a D-dimensional problem there are 2^D possible configurations of the eigenvalue signs, out of which there are only two configurations where all the eigenvalue signs match - one indicates a local minimum (all positive); and one indicates a local maximum (all negative). Therefore, at any point there are $2^D - 2$ possible saddle configurations, and only 1 possible local minimum configuration of the Hessian eigenvalue signs. This shows that as problem dimensionality increases, the number of possible saddle configurations at any point in the search space grows exponentially compared to the number of local minima.

Question 4c - Why can saddle points be harmful to gradient descent? At a saddle point s , the gradient vanishes $\nabla f(s) = 0$. Looking at the gradient descent formula:

$$\mathbf{w}^{\tau+1} = \mathbf{w}^\tau - \eta \nabla f$$

we notice that if the optimizer reaches the saddle point, it will seem as if the search converged and the parameters will not be updated anymore, since there is no gradient information at the saddle point. Furthermore,

$$\lim_{\Delta \rightarrow 0} \mathbf{w}^\tau - \eta \nabla f(s + \Delta) = \mathbf{w}^\tau$$

meaning that as we get closer and closer to the saddle point s , the gradient updates become smaller and smaller, and we risk completely losing the gradient information once we reach the saddle point, even though we are potentially nowhere near a local minimum.

5 Batch Normalization

Batch Normalization first z-scores its inputs $\mathbf{x}$ to produce normalized activations $\hat{\mathbf{x}}$:

$$\hat{\mathbf{x}}_i = \frac{\mathbf{x}_i - \mu_i}{\sigma_i + \epsilon}$$

where μ_i and σ_i represent the mean and standard deviation of each input $\mathbf{x}_i \in \mathbb{R}^S$ across S batch samples, and ϵ is a constant for numerical stability. Finally, batch normalization adds scaling and offset learnable parameters to each normalized activation $\hat{\mathbf{x}}_i \in \mathbb{R}^S$:

$$\mathbf{y}_i = \gamma_i \hat{\mathbf{x}}_i + \beta_i \mathbf{1}_s$$

Question 5a - Gradients of Batch Normalization parameters

$$\begin{aligned} \frac{\partial L}{\partial \gamma_i} &= \sum_s \left[\frac{\partial L}{\partial y_i} \right]_s \hat{x}_{si} & \frac{\partial L}{\partial \gamma} &= \sum_s \left[\frac{\partial L}{\partial \mathbf{Y}} \right]_{s,:} \odot \hat{\mathbf{X}}_{s,:} \\ \frac{\partial L}{\partial \beta_i} &= \sum_s \left[\frac{\partial L}{\partial y_i} \right]_s & \frac{\partial L}{\partial \beta} &= \sum_s \left[\frac{\partial L}{\partial \mathbf{Y}} \right]_{s,:} \end{aligned}$$

Question 5b A fully-connected layer with D inputs and M outputs has $M(D + 1)$ learnable parameters. For each of the M outputs, batch normalization adds 2 learnable parameters γ and β . Thus, a fully-connected layer followed by batch normalization corresponds to $M(D + 1) + 2M$ learnable parameters. In our example, $D = 20, M = 40$. Thus, the fully connected layer contributes $40(20 + 1) = 840$ parameters, while the batch normalization adds $2(40) = 80$ parameters, for a total of 920 learnable parameters.

Question 5c At test time, we want to be flexible with what batch size we use, since we do not know the size of the test dataset ahead of time and we might want to deploy our model online, where it might only receive one sample at a time. In the extreme case when we test on one test sample at a time ($S = 1$), we cannot calculate the mean and variance over the batch, and in any case, with small batch sizes at test times, the estimates would be highly inaccurate.

Batch normalization addresses this issue by keeping a running average of the batch mean and batch variance for each hidden node over the course of training, and stores these values for calculating $\hat{\mathbf{x}}$ at test-time.

Question 5d While ReLU activation function addressed the problem of vanishing gradients, it introduced the issue of dead neurons. Given a zero-centered initialization of model parameters, by definition, half of ReLU inputs will be negative and will result in 0 activation. With increasing depth, the proportion of neurons in a layer that have negative activations for all inputs in the training set increases. This is a problem, because these ReLU 'kills' these neurons and the 'dead neurons' do not learn anything and do not transmit any useful information, neither in the forward, nor backward pass.

Question 5e Batch normalization prevents neurons from dying because it z-scores each neuron’s activations relative to its own activations across the batch. That means that even if a neuron always responds negatively, after z-scoring half of its responses will be positive, and half will be negative. The learnable parameters γ, β for that neuron will then adjust to what is optimal for the network’s performance. For example, if an otherwise ‘dead’ neuron’s input is critical for the network’s performance, the scaling parameter γ might increase and the offset parameter β might shift the z-scored activations to more positive range, to make sure that a large-enough proportion of the neuron’s activations will pass through the ReLU and help the network’s performance.

Question 5f With the same setup as before (single hidden layer of 128 neurons), batch normalization does not affect the results: **test accuracy 50% without vs 49.3% with batch norm**. Different random seeds alternate between original and batch-norm network being marginally better. The final test accuracy also depends on whether we set `model.eval()` before calculating the test accuracy. With `model.train()` batch normalization is performed based on the mean and standard deviation of the test batches, while `model.eval()` uses batch statistics from the training set. Overall, the lack of effect of batch normalization makes sense because in our simple network, there is only one ReLU-like (ELU) nonlinearity. This means there are no compounding effects of ReLU killing increasing number of neurons as the network gets deeper. A single hidden layer MLP is too simple to benefit from Batch Norm and might even slightly suffer from having to fit 256 more parameters in the same number of epochs (although that is largely negligible). Importantly, when keeping all parameters the same and increasing network depth by stacking 128-node hidden layers (Figure 3), we see that batch normalization starts having more and more of an influence, to the point that in very deep networks (depth ≥ 7), the default model without batch normalization fails to learn anything and performs at chance level ($\pm 10\%$ accuracy), while the model with batch normalization work as well as, or better than the 1-hidden layer MLP.

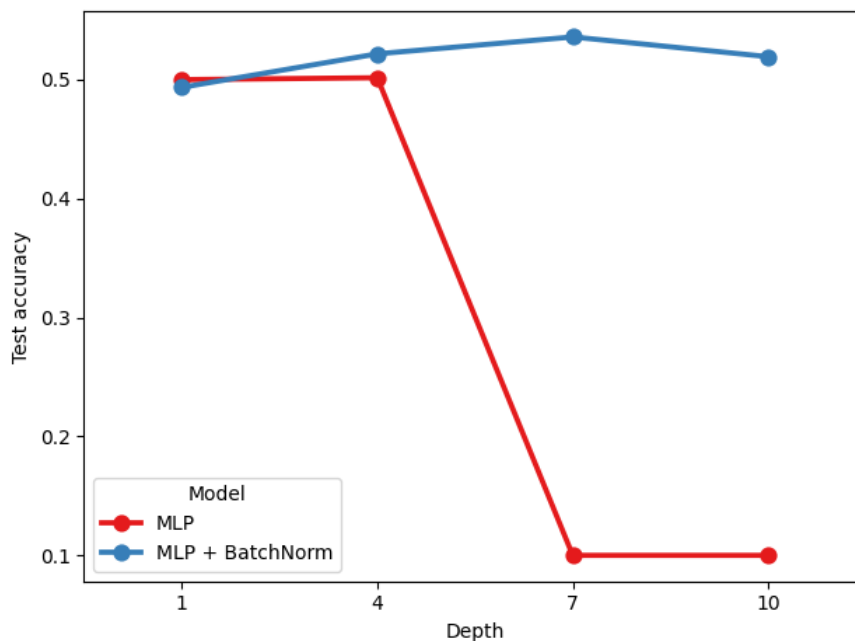


Figure 3: Effect of Batch Norm on test accuracy as function of MLP depth.